

UNIVERSIDAD POLITÉCNICA DE PACHUCA

Ingeniería en Tecnologías de la Información e Innovación Digital

DESARROLLO DE APLICACIONES MÓVILES

Reporte de Práctica

Desarrollo de un Perfil Interactivo con Jetpack Compose y MapLibre

Alumno: Daniel Jimenez Pluma

Grupo: TIID 04 01

Docente: Miguel Angel Montoya Cerro

Lugar: Zempoala, Hidalgo, México

Fecha: 5 de noviembre de 2026

Índice general

1 Introducción **6**

1.1 Contextualización 6

1.2 Planteamiento del problema 6

1.3 Objetivo general 7

1.4 Objetivos específicos 7

1.5 Justificación 7

1.6 Estructura del documento 7

2 Marco teórico **8**

2.1 Android Studio y Kotlin 8

2.2 Jetpack Compose 8

2.3 Material Design 3 8

2.4 Scaffold 9

2.5 Column y Row 9

2.6 Desplazamiento vertical y horizontal 9

2.7 remember y rememberCoroutineScope 9

2.8 Toast 10

2.9 Snackbar 10

2.10 MapLibre 10

2.11 OpenFreeMap 10

2.12 GeoJSON 11

2.13 AndroidView 11

2.14 Resumen de tecnologías utilizadas 11

3 Desarrollo **12**

3.1 Creación y configuración del proyecto 12

3.2 Configuración de MainActivity 12

3.3 Diseño de la pantalla principal 13

3.4 Implementación de la portada 13

3.5 Fotografía de perfil 13

3.6 Información personal 13

3.7 Sección About 14
3.8 Implementación de habilidades e intereses 14
3.9 Desplazamiento horizontal de etiquetas 15
3.10 Interacción mediante Toast 15
3.11 Integración del mapa interactivo 16
3.12 Creación de MapView 16
3.13 Configuración del estilo del mapa 16
3.14 Creación del marcador geográfico 17
3.15 Configuración de la cámara 18
3.16 Controles de acercamiento y alejamiento 18
3.17 Botón para centrar el mapa 18
3.18 Integración de MapLibre con Compose 19
3.19 Administración del ciclo de vida del mapa 19
3.20 Implementación del botón flotante 20
3.21 Implementación de Snackbar 20

4 Evidencias de desarrollo y pruebas 21
4.1 Pantalla principal 21
4.2 Portada y fotografía de perfil 23
4.3 Datos personales 23
4.4 Sección About 24
4.5 Habilidades e intereses 25
4.6 Desplazamiento horizontal 26
4.7 Prueba del mensaje Toast 26
4.8 Visualización del mapa 26
4.9 Prueba de los controles del mapa 27
4.10 Prueba del mensaje Snackbar 27

5 Análisis y argumentación 28

6 Resultados 29

7 Explicación del código 29
7.1 Importaciones principales 29

7.2 Variables de diseño 30
7.3 MainActivity 30
7.4 Configuración de Scaffold 30
7.5 Columna principal 31
7.6 Componente Image 32
7.7 Fotografía circular 32
7.8 Etiquetas de habilidades 33
7.9 Colores de las etiquetas 33
7.10 Surface e interacción Toast 34
7.11 Inicialización de la ubicación 34
7.12 Inicialización de MapView 34
7.13 Estado del mapa 35
7.14 Carga del mapa y estilo 35
7.15 Fuente GeoJSON 36
7.16 Capa del marcador 36
7.17 Cámara del mapa 37
7.18 Controles del mapa 37
7.19 AndroidView 37
7.20 DisposableEffect 38
7.21 FloatingActionButton y Snackbar 39

8 Elementos principales utilizados 40

9 Conclusiones 40

9.1 Aportes y limitaciones 40
9.2 Aprendizajes obtenidos 41

10 Trabajo futuro 41

11 Anexos 41

11.1 Anexo A. Evidencias de funcionamiento 41
11.2 Anexo B. Repositorio del proyecto 42

12 Referencias 42

Índice de ilustraciones

1 Pantalla principal de la aplicación PerfilInteractivo. 22
2 Presentación de la portada y fotografía de perfil. 23
3 Información personal y académica del perfil. 24
4 Sección About con la descripción personal del estudiante. 25
5 Etiquetas de habilidades e intereses tecnológicos. 25
6 Desplazamiento horizontal de las etiquetas. 26
7 Mensaje Toast al seleccionar una etiqueta. 26
8 Mapa interactivo con marcador de ubicación. 27
9 Controles de navegación del mapa interactivo. 27
10 Mensaje Snackbar activado mediante el botón flotante. 28

Índice de tablas

1 Tecnologías y componentes utilizados 11
2 Elementos principales de la aplicación 40

1. Introducción

1.1. Contextualización

El desarrollo de aplicaciones móviles requiere la implementación de interfaces que permitan presentar información de manera organizada, atractiva e intuitiva. Actualmente, Android ofrece herramientas que facilitan la construcción de aplicaciones mediante componentes reutilizables y tecnologías modernas.

Jetpack Compose es un conjunto de herramientas que permite desarrollar interfaces de usuario utilizando Kotlin y un enfoque declarativo. Mediante funciones anotadas con `@Composable`, es posible construir pantallas, organizar elementos visuales y establecer interacciones con el usuario.

En esta práctica se desarrolló una aplicación denominada **PerfilInteractivo**, cuyo propósito es mostrar un perfil personal y académico mediante una interfaz móvil.

La aplicación incorpora imágenes, información personal, descripciones, etiquetas de habilidades e intereses y un mapa interactivo. También integra mensajes emergentes para proporcionar respuestas a las acciones realizadas por el usuario.

Para la representación geográfica se utilizó MapLibre junto con el servicio de estilos de OpenFreeMap, permitiendo visualizar un mapa dentro de la aplicación sin utilizar directamente la API de Google Maps.

1.2. Planteamiento del problema

Una aplicación de perfil personal necesita presentar información de manera clara y permitir que el usuario interactúe con los elementos disponibles. Una pantalla que únicamente contiene textos e imágenes puede resultar limitada si no incorpora elementos que faciliten la exploración del contenido.

Por esta razón, se planteó desarrollar una interfaz que integrara información personal, habilidades tecnológicas y ubicación geográfica, además de incorporar diferentes mecanismos de interacción.

La pregunta que guía esta práctica es cómo construir una aplicación móvil que permita mostrar un perfil completo y dinámico mediante Jetpack Compose, incorporando componentes visuales, mensajes emergentes y un mapa interactivo.

Para resolverlo se utilizaron componentes de Material 3, funciones composables, estados, corrutinas y una biblioteca externa para la visualización cartográfica.

1.3. Objetivo general

Desarrollar una aplicación móvil de perfil personal mediante Kotlin y Jetpack Compose, integrando componentes visuales, elementos interactivos y un mapa geográfico utilizando MapLibre y OpenFreeMap.

1.4. Objetivos específicos

- Construir una interfaz mediante funciones composables.
- Organizar el contenido utilizando componentes de diseño de Jetpack Compose.
- Incorporar imágenes de portada y fotografía de perfil.
- Mostrar información personal, académica y profesional.
- Implementar etiquetas interactivas de habilidades e intereses.
- Utilizar mensajes Toast para responder a la selección de etiquetas.
- Integrar un mapa interactivo mediante MapLibre.
- Implementar controles para acercar, alejar y centrar el mapa.
- Utilizar Snackbar para mostrar mensajes mediante un botón de acción flotante.

1.5. Justificación

La realización de esta práctica permite aplicar conocimientos relacionados con el desarrollo de aplicaciones móviles y la construcción de interfaces mediante Jetpack Compose.

La integración de imágenes, textos, etiquetas y mapas permite comprender cómo diferentes componentes pueden trabajar conjuntamente dentro de una misma pantalla.

Además, la implementación de elementos interactivos permite reforzar el manejo de eventos, estados y funciones de Kotlin.

El uso de MapLibre representa una alternativa para incorporar mapas en aplicaciones Android sin depender directamente de Google Maps.

1.6. Estructura del documento

El presente documento se organiza en diferentes apartados. Primero se presenta el marco teórico de las tecnologías utilizadas. Posteriormente se describe el desarrollo de la práctica mediante sus diferentes componentes y evidencias.

Después se presenta el análisis de la implementación, los resultados obtenidos y la explicación del código por bloques. Finalmente se incluyen las conclusiones, el trabajo futuro, los anexos y las referencias consultadas.

2. Marco teórico

2.1. Android Studio y Kotlin

Android Studio es el entorno de desarrollo integrado utilizado para crear aplicaciones Android. Permite escribir código, administrar recursos, configurar dependencias y ejecutar aplicaciones en emuladores o dispositivos físicos.

Kotlin es un lenguaje de programación moderno utilizado para desarrollar aplicaciones Android. Su sintaxis permite escribir código organizado y facilita el trabajo con programación orientada a objetos y funciones.

En esta práctica se utilizó Kotlin para definir la actividad principal, las funciones composables, las listas de elementos y las interacciones de la aplicación.

2.2. Jetpack Compose

Jetpack Compose es un toolkit moderno de Android que permite construir interfaces de usuario mediante un enfoque declarativo.

La interfaz se define mediante funciones anotadas con `@Composable`, que describen cómo deben mostrarse los elementos en pantalla.

En lugar de depender exclusivamente de archivos XML, Compose permite definir directamente en Kotlin la estructura, apariencia y comportamiento de los componentes.

2.3. Material Design 3

Material Design 3 es un sistema de diseño que proporciona componentes visuales para desarrollar interfaces modernas.

En la aplicación se utilizaron diferentes componentes de Material 3, entre ellos:

- `MaterialTheme`.
- `Scaffold`.
- `FloatingActionButton`.
- `SnackbarHost`.
- `Surface`.

- `Button`.
- `OutlinedButton`.

Estos componentes permiten mantener una estructura visual organizada y proporcionar interacciones al usuario.

2.4. Scaffold

`Scaffold` es un componente de Material 3 que proporciona una estructura general para organizar una pantalla.

En esta práctica se utilizó para establecer el color de fondo, incorporar el contenedor de Snackbar y colocar un botón de acción flotante.

También proporciona un parámetro de contenido que recibe el espacio disponible para organizar los elementos principales de la aplicación.

2.5. Column y Row

`Column` organiza los elementos de manera vertical, mientras que `Row` permite organizarlos horizontalmente.

Ambos componentes se utilizaron para distribuir los elementos del perfil y presentar las etiquetas de habilidades.

La combinación de estos contenedores permite construir interfaces organizadas sin necesidad de definir manualmente la posición de cada elemento.

2.6. Desplazamiento vertical y horizontal

Jetpack Compose proporciona modificadores que permiten desplazar contenido cuando este supera el espacio disponible.

En la práctica se utilizó `verticalScroll` para permitir que el usuario recorriera la pantalla completa.

Por otra parte, `horizontalScroll` se utilizó para desplazar las etiquetas de habilidades e intereses en una fila horizontal.

2.7. remember y rememberCoroutineScope

`remember` permite conservar un valor durante las recomposiciones de una función composable.

En la aplicación se utilizó para mantener la instancia de SnackbarHostState y la referencia de ubicación del mapa.

`rememberCoroutineScope` proporciona un ámbito de corrutinas asociado a la composición, permitiendo ejecutar operaciones suspendidas desde eventos de interacción.

En esta práctica se utilizó para mostrar el mensaje Snackbar cuando el usuario presiona el botón flotante.

2.8. Toast

Toast es un mecanismo de Android que permite mostrar mensajes breves y temporales sobre la interfaz.

En la aplicación se utilizó para mostrar el nombre de la tecnología seleccionada cuando el usuario presiona alguna de las etiquetas de habilidades.

Para obtener el contexto necesario se utilizó `LocalContext.current`.

2.9. Snackbar

Snackbar es un componente de Material 3 que permite mostrar mensajes temporales dentro de una aplicación.

En esta práctica se implementó mediante `SnackbarHost` y `SnackbarHostState`.

El mensaje se activa desde el botón flotante y utiliza una duración larga para permitir que el usuario pueda leerlo.

2.10. MapLibre

MapLibre es una biblioteca de código abierto que permite integrar mapas interactivos en aplicaciones móviles.

En esta práctica se utilizó MapLibre Native para Android, permitiendo crear una vista cartográfica e incorporarla a Jetpack Compose mediante el componente `AndroidView`.

La biblioteca también permite controlar la cámara del mapa, establecer niveles de zoom y agregar elementos visuales sobre el mapa.

2.11. OpenFreeMap

OpenFreeMap proporciona estilos de mapas compatibles con MapLibre.

Para esta aplicación se utilizó el estilo Liberty mediante la dirección:

https://tiles.openfreemap.org/styles/liberty

Este estilo permite representar calles, zonas geográficas y diferentes elementos cartográficos dentro de la aplicación.

2.12. GeoJSON

GeoJSON es un formato utilizado para representar información geográfica mediante estructuras de datos.

En la práctica se utilizó para crear una fuente de datos que representa la ubicación seleccionada mediante un punto.

La información se genera utilizando `Feature.fromGeometry()` y `Point.fromLngLat()`.

2.13. AndroidView

`AndroidView` permite incorporar vistas tradicionales de Android dentro de una interfaz construida con Jetpack Compose.

En esta práctica se utilizó para mostrar el componente `MapView` de MapLibre dentro de la pantalla principal.

2.14. Resumen de tecnologías utilizadas

Tabla 1: Tecnologías y componentes utilizados

Tecnología	**Función dentro del proyecto**
Kotlin	Lenguaje de programación principal.
Jetpack Compose	Construcción de la interfaz de usuario.
Material 3	Componentes visuales y estructura de pantalla.
Scaffold	Organización general de la pantalla.
Toast	Mensajes al seleccionar etiquetas.
Snackbar	Mensaje activado por el botón flotante.
MapLibre	Visualización del mapa interactivo.
OpenFreeMap	Estilo cartográfico Liberty.
GeoJSON	Representación de la ubicación mediante un punto.
AndroidView	Integración de MapView en Compose.

3. Desarrollo

3.1. Creación y configuración del proyecto

Para desarrollar la aplicación se trabajó con un proyecto Android denominado `PerfilInteractivo`, utilizando Kotlin y Jetpack Compose.

El paquete principal del proyecto es:

`com.example.perfilinteractivo`

La actividad principal se encuentra en el archivo `MainActivity.kt`, desde donde se configura la actividad y se establece la pantalla inicial.

3.2. Configuración de MainActivity

La clase `MainActivity` hereda de `ComponentActivity` y sobrescribe el método `onCreate()`.

Dentro de este método se configura la biblioteca osmdroid, estableciendo sus preferencias y el valor del agente de usuario.

Posteriormente se establece la interfaz principal mediante `setContent()` y `MaterialTheme`.

Aunque se conserva esta configuración de osmdroid en el archivo, la vista del mapa que se presenta al usuario se implementa mediante MapLibre.

```
1   class MainActivity : ComponentActivity() {
2
3       override fun onCreate(savedInstanceState: Bundle?) {
4           super.onCreate(savedInstanceState)
5
6           Configuration.getInstance().load(
7           applicationContext,
8           getSharedPreferences("osmdroid", MODE_PRIVATE)
9           )
10
11          Configuration.getInstance().userAgentValue =
12          "PerfilInteractivo/1.0 (com.example.perfilinteractivo)"
13
14          setContent {
15              MaterialTheme {
16                  PantallaPerfil()
17              }
```

```
18                 }
19             }
20         }
21
```

3.3. Diseño de la pantalla principal

La función `PantallaPerfil()` contiene la estructura principal de la aplicación.

Se utilizó `Scaffold` para organizar el contenido, establecer el color de fondo y proporcionar un espacio para mostrar mensajes mediante Snackbar.

También se incorporó un botón de acción flotante que permite ejecutar una acción cuando el usuario lo presiona.

El contenido principal se organiza mediante una columna vertical que permite desplazarse por toda la pantalla.

3.4. Implementación de la portada

La parte superior de la aplicación contiene una imagen de portada que ocupa todo el ancho disponible.

Para incorporarla se utilizó el componente `Image` con el recurso:

`R.drawable.portada`

La imagen tiene una altura de 180 dp y utiliza `ContentScale.Crop` para adaptarse al espacio establecido.

3.5. Fotografía de perfil

Debajo de la portada se presenta una fotografía personal utilizando el recurso:

`R.drawable.img2`

La imagen tiene un tamaño de 112 dp y utiliza `CircleShape` para obtener una presentación circular.

Además, se emplea `ContentScale.Crop` para adaptar la fotografía al contenedor.

3.6. Información personal

La aplicación presenta el nombre del estudiante, su correo electrónico y su ocupación.

Los datos se distribuyen mediante componentes `Column` y `Text`, utilizando diferentes tamaños, colores y pesos tipográficos.

La información mostrada es la siguiente:

- Nombre: Daniel Jimenez Pluma.
- Correo electrónico: danieljip02@gmail.com.
- Ocupación: Estudiante de Ingeniería.
- Área: Desarrollador Android.

3.7. Sección About

La sección About contiene una descripción personal y académica del estudiante.

En ella se presentan los estudios de Ingeniería en Tecnologías de la Información e Innovación Digital en la Universidad Politécnica de Pachuca.

También se describen los conocimientos relacionados con el desarrollo de software, incluyendo Kotlin, Jetpack Compose, Java, C++, PHP, Laravel y JavaScript.

Se mencionan herramientas y tecnologías como MongoDB, PostgreSQL, Linux, Docker, Git y GitHub.

Finalmente, se incorporó una descripción humorística relacionada con Batman, haciendo referencia a una identidad secreta y a la programación.

3.8. Implementación de habilidades e intereses

Se desarrolló la función composable `EtiquetasPerfil()`, encargada de mostrar las habilidades e intereses del estudiante.

Se creó una lista con diez elementos:

```
1         val etiquetas = listOf(
2         "Kotlin",
3         "Jetpack Compose",
4         "Java",
5         "C++",
6         "PHP / Laravel",
7         "JavaScript",
8         "MongoDB",
9         "PostgreSQL",
10        "Linux / Docker",
11        "Batman"
```

```
        )

```

Cada elemento se presenta mediante un componente `Surface`, que permite establecer el color, el borde y la forma de las etiquetas.

Para diferenciar visualmente cada elemento se definió una lista de colores que se reutiliza mediante el índice de cada etiqueta.

3.9. Desplazamiento horizontal de etiquetas

Debido a que las diez etiquetas no caben en una sola fila dentro de la pantalla, se implementó el modificador `horizontalScroll()`.

Este modificador permite que el usuario desplace horizontalmente la fila y consulte todas las etiquetas.

```
        Row(
        modifier = Modifier
        .fillMaxWidth()
        .horizontalScroll(rememberScrollState())
        .padding(
        horizontal = 20.dp,
        vertical = 14.dp
        ),
        horizontalArrangement = Arrangement.spacedBy(9.dp)
        )

```

De esta manera se conserva una distribución horizontal sin reducir el tamaño de los elementos.

3.10. Interacción mediante Toast

Cada etiqueta permite ejecutar una acción cuando el usuario la selecciona.

Para obtener el contexto de Android se utiliza:

```
        val context = LocalContext.current

```

Posteriormente se utiliza `Toast.makeText()` para mostrar un mensaje con el nombre de la tecnología seleccionada.

```
        Toast.makeText(
        context,
```

```
        "Tecnologia: $etiqueta",
        Toast.LENGTH_SHORT
        ).show()

```

Esto permite proporcionar una respuesta inmediata a las acciones realizadas sobre las etiquetas.

3.11. Integración del mapa interactivo

Para incorporar la ubicación geográfica se utilizó la biblioteca MapLibre.

La ubicación se define mediante una instancia de `LatLng`:

```
        val ubicacion = remember {
            LatLng(19.9806, -98.6847)
        }

```

Las coordenadas permiten establecer el punto geográfico que será utilizado como centro del mapa.

3.12. Creación de MapView

Se utiliza `MapView` para construir la vista cartográfica de MapLibre.

La instancia se conserva mediante **`remember`** y se inicializa utilizando `MapLibre.getInstance()`.

```
        val mapView = remember(context) {
            MapLibre.getInstance(context)

            MapView(context).apply {
                onCreate(null)
            }
        }

```

La vista se incorpora posteriormente a la interfaz mediante el componente `AndroidView`.

3.13. Configuración del estilo del mapa

Para mostrar la información geográfica se utiliza el estilo Liberty proporcionado por OpenFreeMap.

```
        mapa.setStyle(
        Style.Builder().fromUri(
        "https://tiles.openfreemap.org/styles/liberty"
        )
        ) { estilo ->

            // Configuracion de la fuente y marcador
        }

```

El estilo permite visualizar el mapa con información cartográfica y diferentes elementos geográficos.

3.14. Creación del marcador geográfico

Para representar la ubicación se creó una fuente GeoJSON mediante `GeoJsonSource`.

La fuente contiene una geometría de tipo punto que utiliza las coordenadas definidas anteriormente.

```
        val fuente = GeoJsonSource(
        "ubicacion-source",
        Feature.fromGeometry(
        Point.fromLngLat(
        ubicacion.longitude,
        ubicacion.latitude
        )
        )
        )

        estilo.addSource(fuente)

```

Posteriormente se creó una capa visual mediante `CircleLayer`.

```
        val marcador = CircleLayer(
        "ubicacion-layer",
        "ubicacion-source"
        ).withProperties(
        PropertyFactory.circleRadius(9f),
        PropertyFactory.circleColor("#D32F2F"),
        PropertyFactory.circleStrokeColor("#FFFFFF"),
        PropertyFactory.circleStrokeWidth(2f)
```

```
        )

        estilo.addLayer(marcador)

```

El marcador utiliza un círculo rojo con borde blanco, lo que permite identificar visualmente el punto seleccionado.

3.15. Configuración de la cámara

La cámara del mapa se configura mediante `CameraPosition.Builder()`.

Se establece como objetivo la ubicación definida anteriormente y se asigna un nivel de zoom de 15.

```
        mapa.cameraPosition =
        CameraPosition.Builder()
        .target(ubicacion)
        .zoom(15.0)
        .build()

```

Esto permite que el mapa se muestre inicialmente centrado en la ubicación establecida.

3.16. Controles de acercamiento y alejamiento

Se incorporaron dos botones para modificar el nivel de acercamiento del mapa.

El botón de alejar utiliza:

```
        CameraUpdateFactory.zoomOut()

```

El botón de acercar utiliza:

```
        CameraUpdateFactory.zoomIn()

```

Ambos controles utilizan `animateCamera()` para realizar el cambio de manera animada.

3.17. Botón para centrar el mapa

También se incorporó un botón denominado `Centrar`, cuya función es regresar la cámara a la ubicación inicial.

Para ello se crea una nueva posición de cámara utilizando las coordenadas originales y un zoom de 15.

```
        mapaCargado?.let {
            it.animateCamera(
            CameraUpdateFactory.newCameraPosition(
            CameraPosition.Builder()
            .target(ubicacion)
            .zoom(15.0)
            .build()
            )
            )
        }

```

El operador seguro permite ejecutar la acción únicamente cuando la instancia del mapa se encuentra disponible.

3.18. Integración de MapLibre con Compose

Para mostrar el mapa dentro de la interfaz se utilizó `AndroidView`.

```
        AndroidView(
        factory = { mapView },
        modifier = Modifier
        .fillMaxWidth()
        .height(280.dp)
        .clip(RoundedCornerShape(16.dp))
        )

```

Este componente permite utilizar la vista tradicional de Android dentro de la estructura de Jetpack Compose.

Se estableció una altura de 280 dp y bordes redondeados para mantener la apariencia visual de la aplicación.

3.19. Administración del ciclo de vida del mapa

MapLibre necesita administrar diferentes eventos relacionados con el ciclo de vida de su vista.

Para ello se utilizó `DisposableEffect`, que permite ejecutar acciones al iniciar y liberar recursos cuando el efecto deja de estar activo.

En la implementación se utilizan los métodos:

- `onStart()`.
- `onResume()`.
- `onPause()`.
- `onStop()`.
- `onDestroy()`.

Esto permite gestionar la vista del mapa durante la composición y su eliminación.

3.20. Implementación del botón flotante

Dentro del componente `Scaffold` se agregó un `FloatingActionButton`.

El botón utiliza el color azul definido mediante la variable `Azul` y presenta un icono de suma.

```
        FloatingActionButton(
        onClick = {
            scope.launch {
                snackbarHostState.showSnackbar(
                message = "¡El caballero de la noche y su práctica de Android!",
                withDismissAction = true,
                duration = SnackbarDuration.Long
                )
            }
        },
        containerColor = Azul,
        contentColor = Color.White,
        shape = CircleShape
        )

```

Al presionar el botón se ejecuta una corrutina que permite mostrar un mensaje mediante Snackbar.

3.21. Implementación de Snackbar

Para mostrar el mensaje se utilizó `SnackbarHostState` y `SnackbarHost`.

El estado se conserva mediante `remember`, mientras que `rememberCoroutineScope()` permite ejecutar la operación suspendida desde el evento del botón flotante.

El mensaje implementado hace referencia al tema de Batman, manteniendo la personalización del perfil.

4. Evidencias de desarrollo y pruebas

En este apartado se presentan las evidencias visuales obtenidas durante la ejecución de la aplicación.

Las capturas permiten comprobar la presentación de los diferentes componentes y el funcionamiento de las interacciones implementadas.

4.1. Pantalla principal

La primera evidencia muestra la aplicación ejecutándose y permite observar la distribución general del perfil, incluyendo los elementos visuales que conforman la pantalla principal.



Ilustración 1: Pantalla principal de la aplicación PerfilInteractivo.

4.2. Portada y fotografía de perfil

En esta evidencia se observa la imagen de portada y la fotografía de perfil.

Se comprueba la correcta incorporación de los recursos gráficos y la aplicación de los modificadores de tamaño, recorte y forma circular.



Ilustración 2: Presentación de la portada y fotografía de perfil.

4.3. Datos personales

La tercera evidencia muestra el nombre, correo electrónico y ocupación del estudiante.

Se comprueba la organización de los textos y su alineación dentro de la pantalla.



Ilustración 3: Información personal y académica del perfil.

4.4. Sección About

En esta captura se observa la descripción personal y académica del estudiante.

Se presentan los conocimientos tecnológicos, herramientas utilizadas y la descripción personalizada relacionada con Batman.



Ilustración 4: Sección About con la descripción personal del estudiante.

4.5. Habilidades e intereses

La quinta evidencia muestra las etiquetas que representan las habilidades e intereses del estudiante.

Se observa la presentación visual de los elementos mediante diferentes colores, bordes y formas redondeadas.

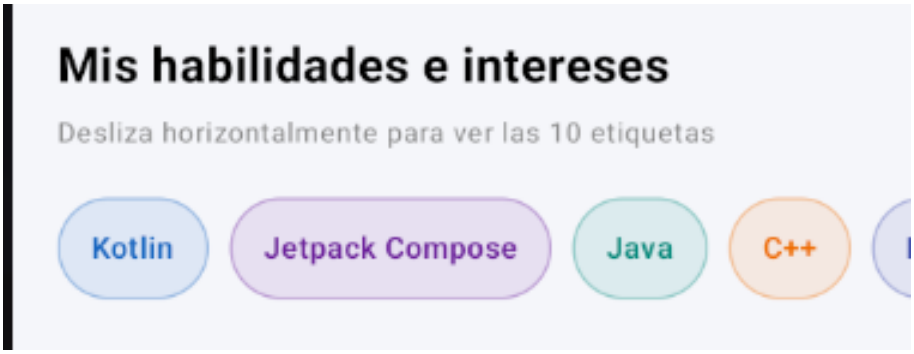


Ilustración 5: Etiquetas de habilidades e intereses tecnológicos.

4.6. Desplazamiento horizontal

En esta evidencia se comprueba el funcionamiento del desplazamiento horizontal de las etiquetas.

La interacción permite consultar los diez elementos sin necesidad de reducir su tamaño para que todos aparezcan simultáneamente en pantalla.

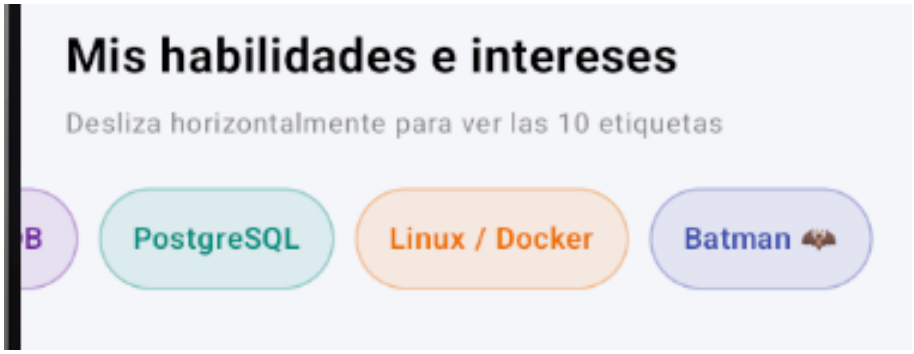


Ilustración 6: Desplazamiento horizontal de las etiquetas.

4.7. Prueba del mensaje Toast

La séptima evidencia muestra el mensaje Toast generado al seleccionar una etiqueta.

Se comprueba que la aplicación identifica el elemento seleccionado y presenta su nombre mediante un mensaje temporal.

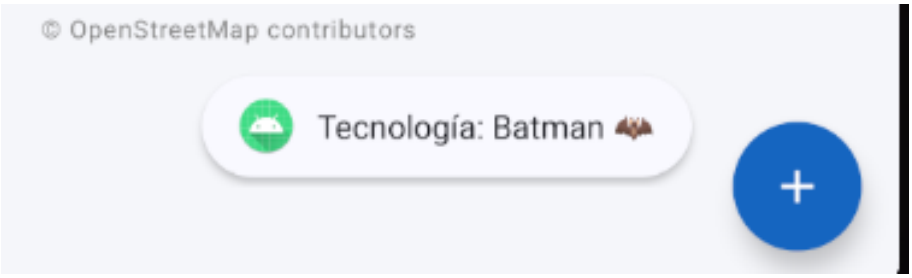


Ilustración 7: Mensaje Toast al seleccionar una etiqueta.

4.8. Visualización del mapa

La octava evidencia muestra el mapa integrado mediante MapLibre y el estilo Liberty de OpenFreeMap.

Se comprueba la carga de la vista cartográfica y la representación del marcador correspondiente a la ubicación establecida.



Ilustración 8: Mapa interactivo con marcador de ubicación.

4.9. Prueba de los controles del mapa

En esta captura se muestran los controles de acercamiento, alejamiento y centrado.

Estos botones permiten modificar la vista del mapa mediante las funciones proporcionadas por MapLibre.



Ilustración 9: Controles de navegación del mapa interactivo.

4.10. Prueba del mensaje Snackbar

La décima evidencia muestra el mensaje que aparece al presionar el botón de acción flotante.

Se comprueba el funcionamiento de Snackbar y la respuesta de la aplicación ante

la interacción del usuario.

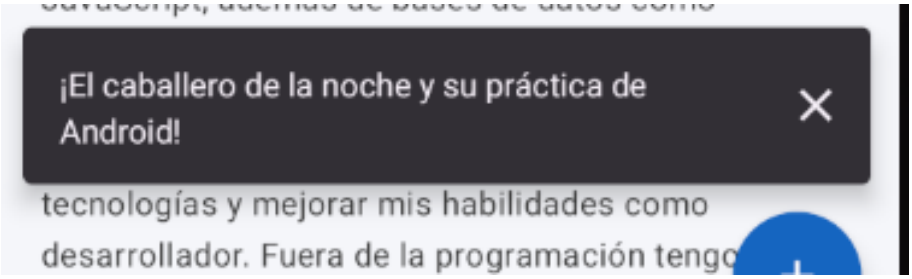


Ilustración 10: Mensaje Snackbar activado mediante el botón flotante.

5. Análisis y argumentación

El desarrollo de PerfilInteractivo permitió integrar diferentes componentes de Jetpack Compose dentro de una misma pantalla.

Uno de los aspectos principales fue la utilización de `Scaffold`, que permitió organizar el contenido y establecer un espacio para el botón flotante y los mensajes Snackbar.

El uso de `Column` y `verticalScroll` facilitó la distribución vertical de los elementos y permitió consultar el contenido completo mediante desplazamiento.

Por otra parte, la implementación de `Row` con `horizontalScroll` permitió mostrar las diez etiquetas de habilidades e intereses sin sobrecargar visualmente la pantalla.

Respecto a la interacción, se comprobó que las etiquetas pueden responder a las acciones del usuario mediante mensajes Toast. Esto demuestra la posibilidad de asociar eventos a los componentes visuales de Compose.

La integración del mapa representó una parte importante del desarrollo, debido a que fue necesario combinar una biblioteca externa con la interfaz declarativa de Jetpack Compose.

Para resolverlo se utilizó `AndroidView`, permitiendo incorporar `MapView` dentro de la composición.

También se implementaron fuentes GeoJSON y una capa de tipo círculo para representar visualmente la ubicación establecida.

Los controles del mapa demostraron que es posible modificar la cámara mediante eventos de interacción, utilizando las funciones proporcionadas por MapLibre.

Finalmente, el botón flotante permitió implementar una acción adicional mediante una corrutina y `SnackbarHostState`.

En conjunto, la práctica permitió comprender cómo integrar componentes visuales, eventos, estados y bibliotecas externas dentro de una aplicación Android desarrollada con Kotlin.

6. Resultados

Como resultado se obtuvo una aplicación móvil denominada PerfilInteractivo, desarrollada mediante Kotlin y Jetpack Compose.

La aplicación presenta un perfil personal con imágenes, información académica, descripción, habilidades tecnológicas y un apartado de ubicación geográfica.

Se incorporaron diez etiquetas interactivas que permiten mostrar mensajes Toast al ser seleccionadas.

También se integró un mapa mediante MapLibre y OpenFreeMap, acompañado de controles que permiten acercar, alejar y centrar la vista.

Finalmente, se implementó un botón de acción flotante que muestra un mensaje mediante Snackbar.

Las evidencias presentadas permiten documentar visualmente los principales componentes y las interacciones desarrolladas durante la práctica.

7. Explicación del código

En este apartado se describen los bloques principales del archivo `MainActivity.kt`, explicando la función de los componentes y las instrucciones más importantes utilizadas en la implementación.

7.1. Importaciones principales

```
1        import androidx.activity.ComponentActivity
2        import androidx.activity.compose.setContent
3        import androidx.compose.foundation.Image
4        import androidx.compose.foundation.layout.*
5        import androidx.compose.material3.*
6        import androidx.compose.runtime.*
7        import androidx.compose.ui.platform.LocalContext
8        import androidx.compose.ui.viewinterop.AndroidView
9        import kotlinx.coroutines.launch
10
```

Descripción.

Estas importaciones permiten utilizar los componentes principales de Android y Jetpack Compose.

También se incluyen las bibliotecas necesarias para mostrar imágenes, organizar elementos, manejar estados, obtener el contexto y utilizar vistas tradicionales de Android.

7.2. Variables de diseño

```
1         private val Azul = Color(0xFF1565C0)
2         private val Fondo = Color(0xFFF5F6FA)
3
```

Descripción.

Se definieron dos variables de tipo Color para mantener una apariencia visual consistente.

La variable `Azul` representa el color principal de los botones y elementos destacados.

La variable `Fondo` establece el color de fondo general de la pantalla.

7.3. MainActivity

```
1         class MainActivity : ComponentActivity() {
2
3             override fun onCreate(savedInstanceState: Bundle?) {
4                 super.onCreate(savedInstanceState)
5
6                 setContent {
7                     MaterialTheme {
8                         PantallaPerfil()
9                     }
10                }
11            }
12        }
13
```

Descripción.

La clase `MainActivity` representa la actividad principal de Android.

El método `onCreate()` se ejecuta al crear la actividad y permite inicializar los componentes necesarios.

Mediante `setContent()` se establece la interfaz de usuario, mientras que `MaterialTheme` proporciona el contexto de diseño para los componentes de Material 3.

7.4. Configuración de Scaffold

```
1         val snackbarHostState = remember {
2             SnackbarHostState()
```

```
        }

        val scope = rememberCoroutineScope()

        Scaffold(
        containerColor = Fondo,
        snackbarHost = {
            SnackbarHost(
            hostState = snackbarHostState
            )
        }
        )

```

Descripción.

Se crea una instancia de `SnackbarHostState` para controlar los mensajes que aparecen en pantalla.

El estado se conserva mediante `remember`, evitando que se reinicialice durante las recomposiciones.

Por otra parte, `rememberCoroutineScope()` proporciona un ámbito para ejecutar la operación suspendida que muestra el mensaje.

El componente `Scaffold` organiza la estructura general de la pantalla y permite establecer el color de fondo y el contenedor de Snackbar.

7.5. Columna principal

```
        Column(
        modifier = Modifier
        .fillMaxSize()
        .padding(padding)
        .background(Fondo)
        .verticalScroll(rememberScrollState())
        )

```

Descripción.

El componente `Column` permite organizar los elementos de manera vertical.

El modificador `fillMaxSize()` ocupa el espacio disponible, mientras que `padding(padding)` aplica el espacio proporcionado por Scaffold.

El modificador `background()` establece el color de fondo y `verticalScroll()` permite desplazar el contenido de la pantalla.

7.6. Componente Image

```
        Image(
        painter = painterResource(
        id = R.drawable.portada
        ),
        contentDescription = "Portada tecnologica",
        modifier = Modifier.fillMaxSize(),
        contentScale = ContentScale.Crop
        )

```

Descripción.

El componente `Image` permite mostrar una imagen almacenada en los recursos de la aplicación.

El recurso se obtiene mediante `painterResource()`.

El modificador `fillMaxSize()` permite ocupar el espacio disponible y `ContentScale.Crop` adapta la imagen al contenedor.

7.7. Fotografía circular

```
        Image(
        painter = painterResource(
        id = R.drawable.img2
        ),
        contentDescription = "Fotografia de perfil",
        modifier = Modifier
        .size(112.dp)
        .align(Alignment.CenterHorizontally)
        .clip(CircleShape),
        contentScale = ContentScale.Crop
        )

```

Descripción.

Se utiliza el recurso `img2` para mostrar la fotografía de perfil.

El modificador `size()` establece sus dimensiones, `align()` permite centrarla horizontalmente y `clip(CircleShape)` genera su forma circular.

7.8. Etiquetas de habilidades

```
        val etiquetas = listOf(
        "Kotlin",
        "Jetpack Compose",
        "Java",
        "C++",
        "PHP / Laravel",
        "JavaScript",
        "MongoDB",
        "PostgreSQL",
        "Linux / Docker",
        "Batman"
        )

```

Descripción.

Se utiliza `listOf()` para crear una lista inmutable que contiene las diez etiquetas.

Posteriormente se recorre mediante `forEachIndexed()`, permitiendo obtener el índice y el texto de cada elemento.

7.9. Colores de las etiquetas

```
        val colores = listOf(
        Color(0xFF1565C0),
        Color(0xFF6A1B9A),
        Color(0xFF00897B),
        Color(0xFFEF6C00),
        Color(0xFF3949AB)
        )

```

Descripción.

Se define una lista de colores que permite diferenciar visualmente las etiquetas.

Mediante el operador módulo se reutilizan los colores cuando el número de etiquetas supera la cantidad de colores disponibles.

7.10. Surface e interacción Toast

```
        Surface(
        onClick = {
            Toast.makeText(
            context,
            "Tecnologia: $etiqueta",
            Toast.LENGTH_SHORT
            ).show()
        },
        color = color.copy(alpha = 0.10f),
        shape = RoundedCornerShape(50.dp)
        )

```

Descripción.

El componente `Surface` permite crear cada etiqueta con una apariencia visual personalizada.

El parámetro `onClick` establece la acción que se ejecuta cuando el usuario selecciona una etiqueta.

Dentro de esta acción se utiliza `Toast.makeText()` para mostrar un mensaje temporal con el nombre del elemento seleccionado.

7.11. Inicialización de la ubicación

```
        val ubicacion = remember {
            LatLng(19.9806, -98.6847)
        }

```

Descripción.

Se crea una instancia de `LatLng` para representar las coordenadas geográficas de la ubicación.

El uso de `remember` permite conservar la instancia durante las recomposiciones.

7.12. Inicialización de MapView

```
        val mapView = remember(context) {
            MapLibre.getInstance(context)

```

```
            MapView(context).apply {
                onCreate(null)
            }
        }

```

Descripción.

Se inicializa MapLibre mediante el contexto de Android y posteriormente se crea una instancia de `MapView`.

El método `onCreate()` inicializa la vista del mapa y permite preparar su funcionamiento.

7.13. Estado del mapa

```
        var mapaCargado by remember {
            mutableStateOf<MapLibreMap?>(null)
        }

```

Descripción.

Se declara una variable de estado que almacena la instancia del mapa cuando está disponible.

Se utiliza `mutableStateOf()` para mantener un valor observable y permitir que Compose actualice los elementos dependientes del estado.

El operador `by` permite acceder directamente al valor de la variable.

7.14. Carga del mapa y estilo

```
        mapa.setStyle(
        Style.Builder().fromUri(
        "https://tiles.openfreemap.org/styles/liberty"
        )
        ) { estilo ->
            // Configuracion de la fuente geografica
        }

```

Descripción.

Se establece el estilo Liberty de OpenFreeMap mediante la dirección del servicio.

Cuando el estilo termina de cargarse, se ejecuta el bloque de código que permite agregar las fuentes y capas necesarias para representar la ubicación.

7.15. Fuente GeoJSON

```
        val fuente = GeoJsonSource(
        "ubicacion-source",
        Feature.fromGeometry(
        Point.fromLngLat(
        ubicacion.longitude,
        ubicacion.latitude
        )
        )
        )

```

Descripción.

Se crea una fuente de información geográfica mediante `GeoJsonSource`.

La geometría se representa como un punto utilizando las coordenadas de longitud y latitud.

Posteriormente la fuente se agrega al estilo del mapa para que pueda ser utilizada por una capa.

7.16. Capa del marcador

```
        val marcador = CircleLayer(
        "ubicacion-layer",
        "ubicacion-source"
        ).withProperties(
        PropertyFactory.circleRadius(9f),
        PropertyFactory.circleColor("#D32F2F"),
        PropertyFactory.circleStrokeColor("#FFFFFF"),
        PropertyFactory.circleStrokeWidth(2f)
        )

```

Descripción.

Se crea una capa de tipo círculo mediante `CircleLayer`.

Se configura el radio del círculo, su color rojo, el color blanco del borde y el grosor del contorno.

Esta capa permite representar visualmente la ubicación establecida.

7.17. Cámara del mapa

```
1        mapa.cameraPosition =
2        CameraPosition.Builder()
3        .target(ubicacion)
4        .zoom(15.0)
5        .build()
6
```

Descripción.

Se establece la posición inicial de la cámara.

El método `target()` define el punto geográfico que se desea visualizar, mientras que `zoom()` establece el nivel de acercamiento inicial.

7.18. Controles del mapa

```
1        mapaCargado?.let {
2            it.animateCamera(
3            CameraUpdateFactory.zoomIn()
4            )
5        }
6
```

Descripción.

Este bloque permite ejecutar una acción sobre el mapa cuando su instancia está disponible.

El operador seguro evita intentar acceder a un objeto nulo.

En este caso, `zoomIn()` aumenta el nivel de acercamiento y `animateCamera()` realiza la transición de manera animada.

El botón de alejar utiliza `zoomOut()`, mientras que el botón centrar establece nuevamente la posición inicial de la cámara.

7.19. AndroidView

```
1        AndroidView(
2        factory = { mapView },
3        modifier = Modifier
4        .fillMaxWidth()
```

```
        .height(280.dp)
        .clip(RoundedCornerShape(16.dp))
        )

```

Descripción.

El componente `AndroidView` permite integrar la vista tradicional de MapLibre dentro de Compose.

Se utiliza `factory` para proporcionar la instancia de `MapView`.

Además, se establece el ancho disponible, una altura de 280 dp y bordes redondeados.

7.20. DisposableEffect

```
        DisposableEffect(mapView) {

            mapView.onStart()
            mapView.onResume()

            mapView.getMapAsync { mapa ->
                mapaCargado = mapa
            }

            onDispose {
                mapaCargado = null
                mapView.onPause()
                mapView.onStop()
                mapView.onDestroy()
            }
        }

```

Descripción.

`DisposableEffect` permite ejecutar operaciones relacionadas con el ciclo de vida de una vista.

Al iniciar el efecto se ejecutan los métodos correspondientes al inicio y reanudación del mapa.

Mediante `getMapAsync()` se obtiene la instancia del mapa cuando está disponible.

Finalmente, `onDispose` permite liberar la instancia y ejecutar los métodos de pausa, detención y destrucción.

7.21. FloatingActionButton y Snackbar

```
        FloatingActionButton(
        onClick = {
            scope.launch {
                snackbarHostState.showSnackbar(
                message = "¡El caballero de la noche y su practica de Android!",
                withDismissAction = true,
                duration = SnackbarDuration.Long
                )
            }
        },
        containerColor = Azul,
        contentColor = Color.White,
        shape = CircleShape
        )

```

Descripción.

Se implementa un botón de acción flotante que utiliza un fondo azul y un icono de suma.

Al presionarlo se ejecuta una corrutina mediante `scope.launch()`.

Dentro de la corrutina se utiliza `showSnackbar()` para mostrar un mensaje temporal en la interfaz.

El parámetro `withDismissAction` permite mostrar una acción para cerrar el mensaje y `SnackbarDuration.Long` establece una duración larga.

8. Elementos principales utilizados

Tabla 2: Elementos principales de la aplicación

Elemento	**Función**
MainActivity	Actividad principal de Android.
PantallaPerfil	Función composable que organiza la pantalla.
Scaffold	Estructura general de la interfaz.
Column	Distribución vertical de los elementos.
Row	Distribución horizontal de las etiquetas y controles.
Image	Presentación de las imágenes del perfil.
Surface	Contenedor visual e interactivo de las etiquetas.
Toast	Mensaje al seleccionar una tecnología.
SnackbarHost	Contenedor de mensajes Snackbar.
FloatingActionButton	Botón flotante para mostrar un mensaje.
remember	Conservación de valores durante recomposiciones.
rememberCoroutineScope	Ejecución de corrutinas desde eventos.
MapView	Vista cartográfica de MapLibre.
AndroidView	Integración de MapView en Compose.
GeoJsonSource	Fuente de datos geográficos.
CircleLayer	Capa visual del marcador.
CameraPosition	Configuración de la cámara del mapa.

9. Conclusiones

9.1. Aportes y limitaciones

La realización de esta práctica permitió reforzar los conocimientos relacionados con el desarrollo de aplicaciones móviles mediante Kotlin y Jetpack Compose.

Se logró construir una interfaz que presenta información personal y académica mediante imágenes, textos y etiquetas interactivas.

También se comprendió la utilidad de los componentes de Material 3 para organizar una pantalla y responder a las acciones realizadas por el usuario.

Uno de los aportes más importantes fue la integración de MapLibre dentro de Jetpack Compose mediante `AndroidView`. Esto permitió incorporar un mapa interactivo con un marcador geográfico y controles para modificar su visualización.

Entre las limitaciones se encuentra que la ubicación se establece mediante coordenadas definidas directamente en el código, por lo que no representa una ubicación obtenida

automáticamente mediante los sensores del dispositivo.

Además, la visualización del mapa depende de la disponibilidad del servicio externo de OpenFreeMap y de una conexión a internet.

9.2. Aprendizajes obtenidos

Durante el desarrollo se reforzó el uso de funciones composables, listas, modificadores, estados y eventos.

También se comprendió la importancia de administrar correctamente el ciclo de vida de las vistas tradicionales cuando se integran dentro de Jetpack Compose.

La práctica permitió observar cómo pueden combinarse bibliotecas externas con componentes nativos para ampliar las funcionalidades de una aplicación Android.

10. Trabajo futuro

Como continuación del proyecto se proponen las siguientes mejoras:

- Incorporar una pantalla de configuración para modificar los datos del perfil.
- Permitir actualizar las imágenes desde la aplicación.
- Agregar nuevas habilidades e intereses.
- Implementar una ubicación dinámica mediante los servicios de localización de Android.
- Agregar marcadores adicionales al mapa.
- Incorporar una barra de navegación inferior.
- Separar los componentes de la interfaz en archivos Kotlin independientes.
- Implementar almacenamiento local para conservar información personalizada.

11. Anexos

11.1. Anexo A. Evidencias de funcionamiento

Las diez ilustraciones presentadas en el apartado de desarrollo constituyen las evidencias visuales de la aplicación PerfilInteractivo.

Cada captura documenta una característica específica de la interfaz o una interacción implementada.

Los archivos utilizados son:

1. `01_pantalla_principal.png`
2. `02_portada_perfil.png`
3. `03_datos_personales.png`
4. `04_seccion_about.png`
5. `05_habilidades.png`
6. `06_etiquetas_desplazamiento.png`
7. `07_mensaje_toast.png`
8. `08_mapa_interactivo.png`
9. `09_controles_mapa.png`
10. `10_mensaje_snackbar.png`

11.2. Anexo B. Repositorio del proyecto

El código fuente de la aplicación se encuentra almacenado en GitHub para facilitar su consulta y seguimiento.

https://github.com/danieljiplu-007/PerfilInteractivo

12. Referencias

- Google. (s. f.). *Jetpack Compose*. Android Developers. https://developer.android.com/compose
- Google. (s. f.). *Diseño de interfaces con Compose*. Android Developers. https://developer.android.com/develop/ui/compose
- Google. (s. f.). *Componentes de Material 3 en Compose*. Android Developers. https://developer.android.com/develop/ui/compose/components
- Google. (s. f.). *State en Jetpack Compose*. Android Developers. https://developer.android.com/develop/ui/compose/state
- JetBrains. (s. f.). *Kotlin documentation*. Kotlin. https://kotlinlang.org/docs/home.html

- MapLibre. (s. f.). *MapLibre Native for Android*. MapLibre. https://maplibre.org/
- OpenFreeMap. (s. f.). *OpenFreeMap*. https://openfreemap.org/
- OpenStreetMap. (s. f.). *OpenStreetMap*. https://www.openstreetmap.org/copyright
- Jiménez Pluma, D. (2026). *PerfilInteractivo* [Repositorio de código]. GitHub. https://github.com/danieljiplu-007/PerfilInteractivo